



TinyML Gesture Recognition Using the XIAO nRF52840 Sense IMU

Part 1: Install the TinyML Software

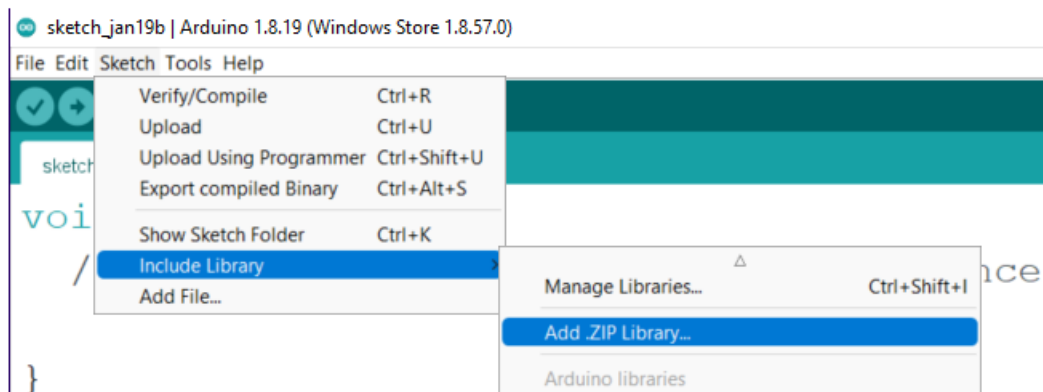
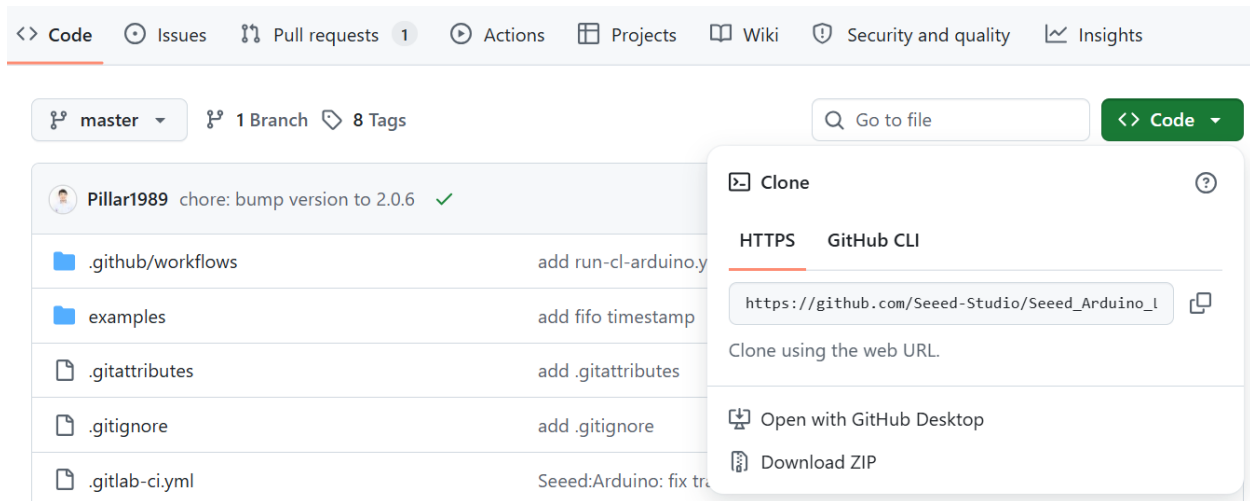
Step 1.1: Confirm the Existing IMU Library

The capture and classifier sketches use:

```
#include <LSM6DS3.h>
LSM6DS3 myIMU(I2C_MODE, 0x6A);
```

Install the Seeed Arduino LSM6DS3 library if it is not already installed. Use the same library that worked in the previous IMU lab. The link is at this link:

https://github.com/Seeed-Studio/Seeed_Arduino_LSM6DS3

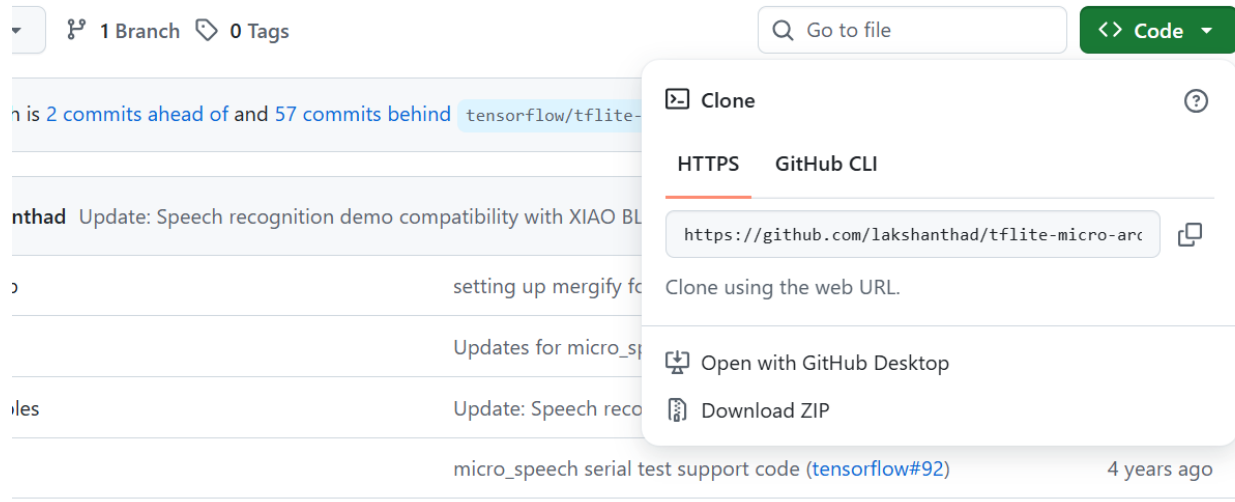


Step 1.2: Install TensorFlow Lite Micro

Follow the Seeed Studio tutorial and install the tflite-micro-arduino-examples library as a ZIP file:

<https://github.com/lakshanthad/tflite-micro-arduino-examples>

1. Download the repository as a ZIP file.
2. In Arduino IDE, select Sketch -> Include Library -> Add .ZIP Library.
3. Select the downloaded ZIP file.
4. Restart Arduino IDE after installation.



Part 2: Capture Gesture Data with IMU_Capture

Step 2.1: Open the Supplied Capture Sketch

Open the following file from the lab package:

```
01_IMU_Capture/IMU_Capture_XIAO.ino
```

The sketch waits for significant motion, then records exactly 119 samples. Each sample contains three accelerometer axes and three gyroscope axes.

Step 2.2: Understand the Required CSV Format

The first Serial Monitor line must be exactly:

```
aX,aY,aZ,gX,gY,gZ
```

Each gesture recording contains 119 data rows. A blank line separates consecutive recordings. Do not change the headings because the training notebook uses these names.

Step 2.3: Upload and Test

1. Upload IMU_Capture_XIAO.ino to the XIAO.
2. Open Serial Monitor and select 115200 baud.
3. Hold the board securely in one hand.
4. Perform a clear punch movement.
5. Confirm that 119 CSV rows appear, followed by a blank line.

```
aX,aY,aZ,gX,gY,gZ  
-0.123,0.041,1.087,-2.350,5.112,0.820  
... 117 additional rows ...  
0.214,-0.078,0.956,1.120,-3.470,0.310
```

Step 2.4: Collect punch.csv

1. Clear or reopen Serial Monitor so the file contains only the required CSV output.
2. Perform at least 10 punches. Twenty recordings are recommended.
3. Use approximately the same board orientation for each punch.
4. Copy the CSV heading and all complete data rows into a plain-text file.
5. Save the file as punch.csv.

Step 2.5: Collect flex.csv

1. Reset the board or reopen Serial Monitor to start a clean capture.
2. Perform at least 10 clear inward flex gestures. Twenty recordings are recommended.
3. Copy the heading and all complete data rows.
4. Save the file as flex.csv.

If slow flex gestures do not cross the motion threshold, change `ACCELERATION_THRESHOLD` from 2.5 to 1.5, then collect both classes again using the same setting.

Checkpoint 2

- `punch.csv` and `flex.csv` both begin with `aX,aY,aZ,gX,gY,gZ`.
- Each file contains complete groups of 119 rows.
- No Arduino status messages are mixed into the data unless they begin with `#`.
- Both files contain multiple gesture recordings.

Part 3: Train the Neural Network in Colab

Step 3.1: Open the Training Notebook

Students may use either the supplied notebook file or the instructor Colab link:

<https://colab.research.google.com/drive/13-4FFAPmWTxryLl8g6Lh7GiJK2mR09Yt?usp=sharing>

Step 3.2: Upload the CSV Files

1. Run the upload cell.
2. Select punch.csv and flex.csv.
3. Confirm that both filenames appear in the notebook output.
4. Run the validation cell and confirm that complete recordings are detected.

Step 3.3: Understand the Input Tensor

One recording contains 119 samples and 6 sensor values per sample:

`119 samples x 6 features = 714 input values`

The notebook normalizes the values exactly as the Arduino classifier does:

- Acceleration: (-4 g to +4 g) is mapped to (0 to 1).
- Gyroscope: (-2000 deg/s to +2000 deg/s) is mapped to (0 to 1).

Step 3.4: Train the Model

The supplied notebook uses the following dense neural-network structure:

Input: 714 values
Dense: 50 neurons, ReLU
Dense: 15 neurons, ReLU
Output: 2 neurons, Softmax

Part 4: Generate model.h

Step 4.1: Convert the Model

Run the TensorFlow Lite conversion cell. It creates:

`gesture_model.tflite`

The next cell converts the binary model to a C array and creates:

```
model.h
```

Step 4.2: Download the Files

1. Download model.h from Colab.
2. Optionally download gesture_model.tflite for documentation and future use.
3. Keep the new model.h with the classifier sketch attached.

Part 5: Deploy IMU_Classifier on the XIAO

Step 5.1: Prepare the Arduino Sketch Folder

Open the supplied classifier folder:

```
IMU_Classifier/
```

It must contain these two files in the same folder:

```
IMU_Classifier.ino  
model.h
```

The package includes the Seeed example punch/flex model so the classifier can be tested immediately. Replace it with the model.h generated from the student data.

Step 5.2: Upload the Classifier

1. Open IMU_Classifier_XIAO.ino.
2. Confirm that the generated model.h is in the same folder.
3. Compile and upload the sketch.
4. Open Serial Monitor at 115200 baud.

Step 5.4: Test Live Inference

Perform a punch and observe the two output scores:

```
punch: 0.982314  
flex: 0.017686  
Prediction: punch confidence: 0.982
```

Then perform a flex gesture and confirm that the flex score becomes the larger value.

Part 6: Measure Classifier Performance

Step 6.1: Controlled Test

Perform five punch trials and five flex trials. Record the result and confidence reported by the board.

Trial	Actual gesture	Predicted gesture	Confidence	Correct?
1	Punch			
2	Punch			
3	Punch			
4	Punch			
5	Punch			
6	Flex			
7	Flex			
8	Flex			
9	Flex			
10	Flex			

Step 6.2: Calculate Accuracy

Use:

$\text{Accuracy} = \text{correct predictions} / \text{total trials} \times 100\%$

A low result should lead to improved data collection before changing the neural-network architecture. Submission Checklist

- punch.csv and flex.csv
- Screenshot of training/validation plots and test accuracy
- Generated model.h
- Working IMU_Classifier_XIAO.ino
- Completed ten-trial performance table
- Short explanation of one classification error or limitation